

Algoritmi Genetici

Implementazione

Passiamo ora a capire come si implementano i GA.

Al seguente link: <https://github.com/SeSa-Lab/ga-examples> troverete due esempi:

- L'esempio giocattolo dei 5 interruttori visto prima (*binary-square*)
- 8 Regine (*eight-queens*)

Entrambi condividono del codice comune (*ga-core*).

Non entriamo nel dettaglio perché l'implementazione, come sappiamo, è sullo stile del programmatore e questo corso NON è un corso di programmazione.

Vi do un grande consiglio.

Tip: *Mai iniziare ad implementare prima di aver (i) compreso il problema da risolvere e (ii) averlo modellato correttamente.*

Comprendere il problema significa capire qual è l'obiettivo che vogliamo raggiungere. Capito ciò, decidiamo di risolvere il problema con un GA. Magari i GA non sono la scelta perfetta (ci sono altri algoritmi che si prestano meglio), però supponiamo di sì per adesso.

Avendo scelto un GA dobbiamo sciogliere **due grandi nodi**:

- Come codifichiamo gli individui (le soluzioni candidate)?
- Come definiamo la funzione di fitness?

Riprendiamo il solito problema delle 8 regine. Come visto con Hill Climbing, lo vediamo come **problema di minimizzazione del numero di conflitti**.

Algoritmi Genetici

Implementazione - Codifica Individui

Tip: *Mai iniziare ad implementare prima di aver (i) compreso il problema da risolvere e (ii) averlo modellato correttamente.*

Nelle 8 regine le soluzioni candidate sono scacchiere con sopra delle regine. Quindi ha senso dire che un individuo sia un'intera **scacchiera con sopra una disposizione delle 8 regine**.

Bene, ma come rappresentiamo la scacchiera? Come matrice? Potrebbe andare, ma è un po' uno spreco di memoria.

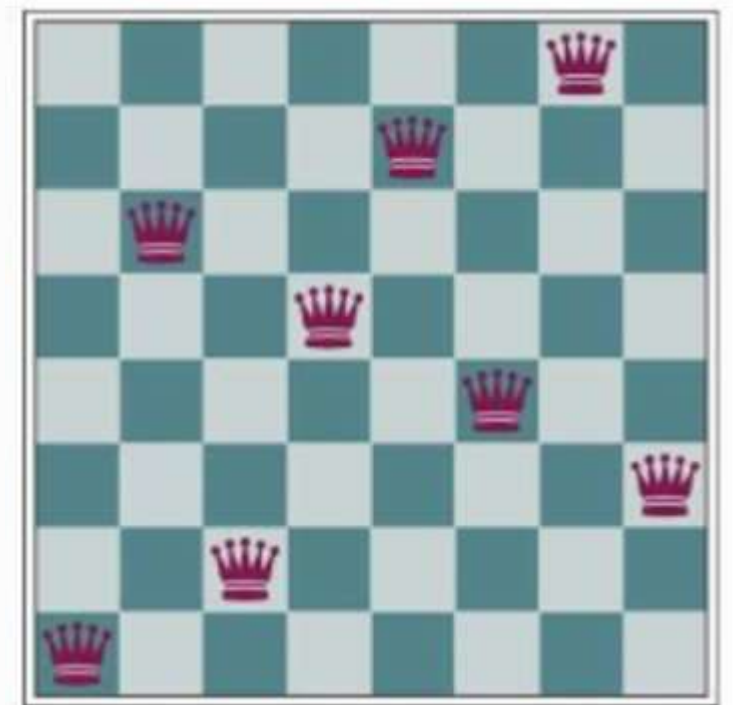
Potremmo usare un array di 8 elementi, dove l'elemento i -esimo rappresenta la colonna i , e il suo valore corrisponde alla riga occupata.

Quindi, la scacchiera in figura è così rappresentata:

(7, 2, 6, 3, 1, 4, 0, 5)

In sostanza, una scacchiera è un **array di interi** di **8 elementi con valori [0, 7]**.

Nel codice, una scacchiera è implementata in *ChessboardIndividual*, sottoclasse di *IntArrayIndividual*. E' qui facciamo la separazione dello spazio delle soluzioni ammissibili e non. Per come è fatta *ChessboardIndividual*, **avremo solo soluzioni ammissibili**.



Algoritmi Genetici

Implementazione - Codifica Popolazione

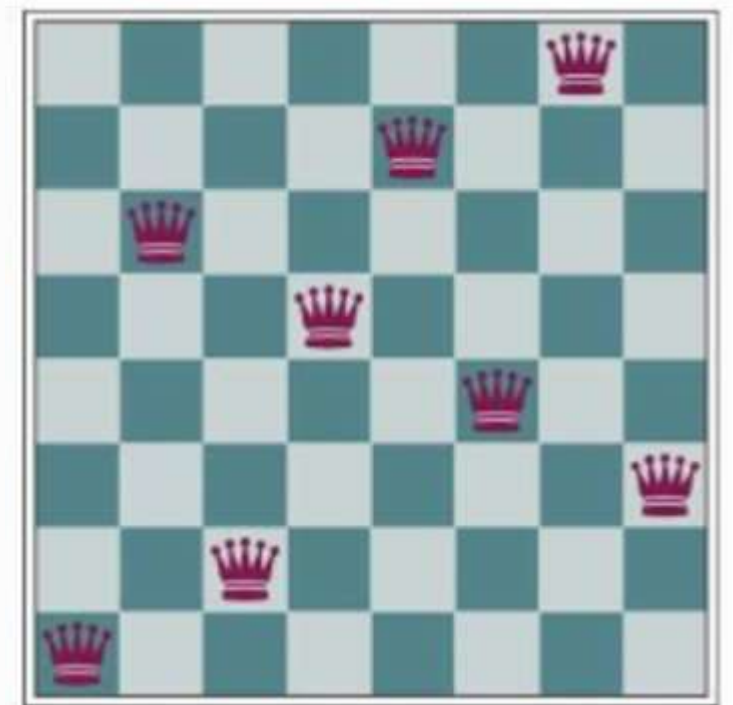
Tip: *Mai iniziare ad implementare prima di aver (i) compreso il problema da risolvere e (ii) averlo modellato correttamente.*

Avendo parlato degli individui, passiamo a dare gli eventuali vincoli della popolazione. Qui ne abbiamo? Dobbiamo far evolvere delle scacchiere, ma sostanzialmente il problema non ci impone altri vincoli particolari.

Quindi, **non abbiamo vincoli per le popolazioni di scacchiere.**

Decidiamo liberamente di usare una popolazione a dimensione fissa (*FixedSizePopulation*).

La prima parte di un GA ci chiede di inizializzare una popolazione. Che cosa usiamo? La cosa più semplice: **inizializzazione casuale**, dove generiamo n scacchiere (ammissibili) casualmente, quindi con un numero ignoto di conflitti (*FixedSizeChessboardRandomInitializer*).



Algoritmi Genetici

Implementazione - Fitness

Tip: *Mai iniziare ad implementare prima di aver (i) compreso il problema da risolvere e (ii) averlo modellato correttamente.*

Ora dobbiamo modellare la funzione di fitness.

Ricordiamoci il nostro obiettivo: **minimizzare il numero di conflitti**.

Quindi ha senso che la fitness (i) sia **di minimo** (i.e., minore è il suo valore, migliore è l'individuo), (ii) **faccia il conto** del numero di conflitti.

La classe *ConflictsFunction* conta i conflitti di riga e di diagonale.

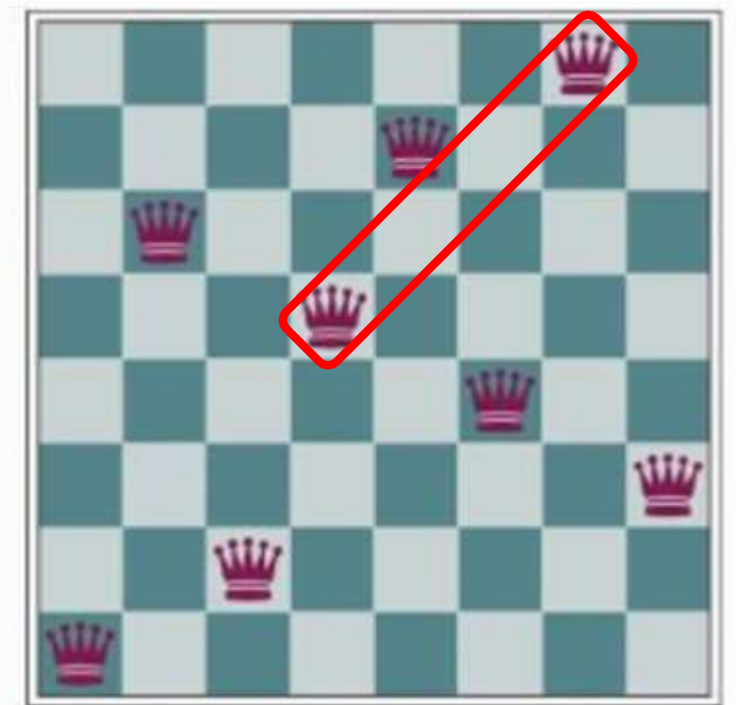
Perché non contiamo i conflitti di colonna?

Per la codifica degli individui scelta, **NON possono** (*by design*) esistere due regine che occupano la medesima colonna (un elemento di un'array può contenere un singolo valore!).

Quindi, in figura:

- Non abbiamo conflitti di riga
- Abbiamo un solo conflitto diagonale (che lo contiamo una sola volta dato che *ConflictFunction* “guarda solo a destra”).

E' un individuo molto buono!



Algoritmi Genetici

Implementazione - Metaeuristica

Tip: *Mai iniziare ad implementare prima di aver (i) compreso il problema da risolvere e (ii) averlo modellato correttamente.*

La parte difficile è fatta, resta da vedere:

- Quale metaeuristica usare?
- Quali operatori genetici usare?
- Preparare una classe *Runner* (i.e., un client che esegue tutto il GA)
- Scegliere un formato di output adeguato

La **metaeuristica** è il nostro “direttore d’orchestra”. Essa dovrebbe occuparsi di:

1. Ricevere in input tutti i parametri necessari
2. Avviare l’inizializzazione della prima popolazione
3. Valutare la prima popolazione rispetto alla data funzione di fitness
4. Avviare il loop di evoluzione e verificare quando terminarlo
5. Restituire i risultati

Nel nostro caso, usiamo *SimpleGeneticAlgorithm*, che implementa un GA semplice (esattamente quello che abbiamo visto finora)

Algoritmi Genetici

Implementazione - Operatori

Tip: *Mai iniziare ad implementare prima di aver (i) compreso il problema da risolvere e (ii) averlo modellato correttamente.*

Ora dobbiamo decidere e poi implementare gli **operatori genetici**.

Il vero problema, in realtà, è decidere quali usare.

- Roulette Wheel o Tournament Selection?
- Single Point o Two Point Crossover?
- Random Resetting o Swap Mutation?

Possiamo seguire diverse idee:

- *Esperienza*. Il fattore più importante. Quanto più si implementano i GA, tanto più ci si rende conto se un certo operatore funziona meglio o peggio in determinati contesti.
- *Letteratura*. Verificare se in letteratura esistono delle analisi comparative degli operatori.
- *Trial-and-error*. Se si è indecisi inizialmente, si può implementare il problema con degli operatori “generalmente buoni” (o anche quelli a noi noti).
Se le prestazioni non ci soddisfano, si cambiano gli operatori e si verifica se ci sono miglioramenti/peggioramenti. E’ la stessa idea che c’è dietro alla selezione ottimale di parametri, ma anche dietro la scelta della metaeuristica.

Banalmente, qui abbiamo scelto *RouletteWheelSelection*, *SinglePointCrossover* e *RandomResettingMutation*.

Algoritmi Genetici

Implementazione - Runner

Tip: *Mai iniziare ad implementare prima di aver (i) compreso il problema da risolvere e (ii) averlo modellato correttamente.*

La classe **Runner** deve sostanzialmente:

- Decidere i parametri (probabilità mutazione, search budget, ecc.)
- Istanziare tutto (funzione fitness, operatori, metaeuristica, ecc.)
- Consegnare tutto alla metaeuristica
- Lanciare la metaeuristica
- Prenderne i risultati

A seconda di come la metaeuristica restituisca i dati, il *Runner*, o qualche altra classe ausiliaria, dovrebbe **formattare l'output**, in un formato human- e/o machine-readable, a seconda di cosa ci torni più utile.

Qui vediamo un esempio di esecuzione dove il miglior individuo ha fitness = 2, ovvero 2 conflitti. Non è l'ottimo globale, ma ci siamo andati vicino...

E' possibile migliorare l'algoritmo?

Ve lo lascio per esercizio!

```
Gen 995) 7.0 vs 2.0 (NewAvg vs BestAvg)
Gen 996) 9.0 vs 2.0 (NewAvg vs BestAvg)
Gen 997) 9.0 vs 2.0 (NewAvg vs BestAvg)
Gen 998) 7.0 vs 2.0 (NewAvg vs BestAvg)
Gen 999) 7.0 vs 2.0 (NewAvg vs BestAvg)
Gen 1000) 5.0 vs 2.0 (NewAvg vs BestAvg)
Search terminated in 1000/1000 iterations.
Best individual is [1, 7, 1, 6, 6, 2, 0, 5], with fitness 2,00.
```

Algoritmi Genetici

Implementazione: E se volessimo usare dei framework?

Assolutamente. Nel materiale aggiuntivo, potete trovare l'implementazione del problema delle 8 regine tramite JMetal: <https://jmetal.github.io/jMetal/>.

Sub-project	Contents
jmetal-core	Core classes
jmetal-solution	Solution encodings
jmetal-algorithm	Algorithm implementations
jmetal-problem	Benchmark problems
jmetal-lab	Experimentation and visualization
jmetal-experimental	New features in development
jmetal-parallel	Parallel extensions
jmetal-auto	Auto-design and configuration
jmetal-component	Component-based algorithms

Già leggendo la documentazione, possiamo osservare diverse cose utili:

- Supporto per la codifica delle soluzioni;
- Supporto al design;
- Supporto all'auto-configurazione degli algoritmi;
- Supporto alla sperimentazione degli algoritmi;
- Supporto alla parallelizzazione.

Inoltre, il framework mette a disposizione diversi algoritmi noti per la risoluzione di problemi multi-obiettivo. Ad esempio, sono implementati gli algoritmi NSGA-II ed NSGA-III, i quali permettono la risoluzione di problemi multi-obiettivo tramite il concetto di “*non-dominating sorting*”: prima di procedere con la selezione, questi algoritmi ordinano gli individui al fine di identificare quelli per cui non esiste alcun altro individuo migliore contemporaneamente per tutti gli obiettivi considerati – in altri termini, cercano di anticipare l'identificazione degli ottimi di Pareto, così da lavorare su individui potenzialmente più interessanti.

Vale la pena notare che JMetal è solo uno dei framework disponibili. Ad esempio:

- <https://pygad.readthedocs.io/en/latest/>
- <https://pypi.org/project/geneticalgorithm/>

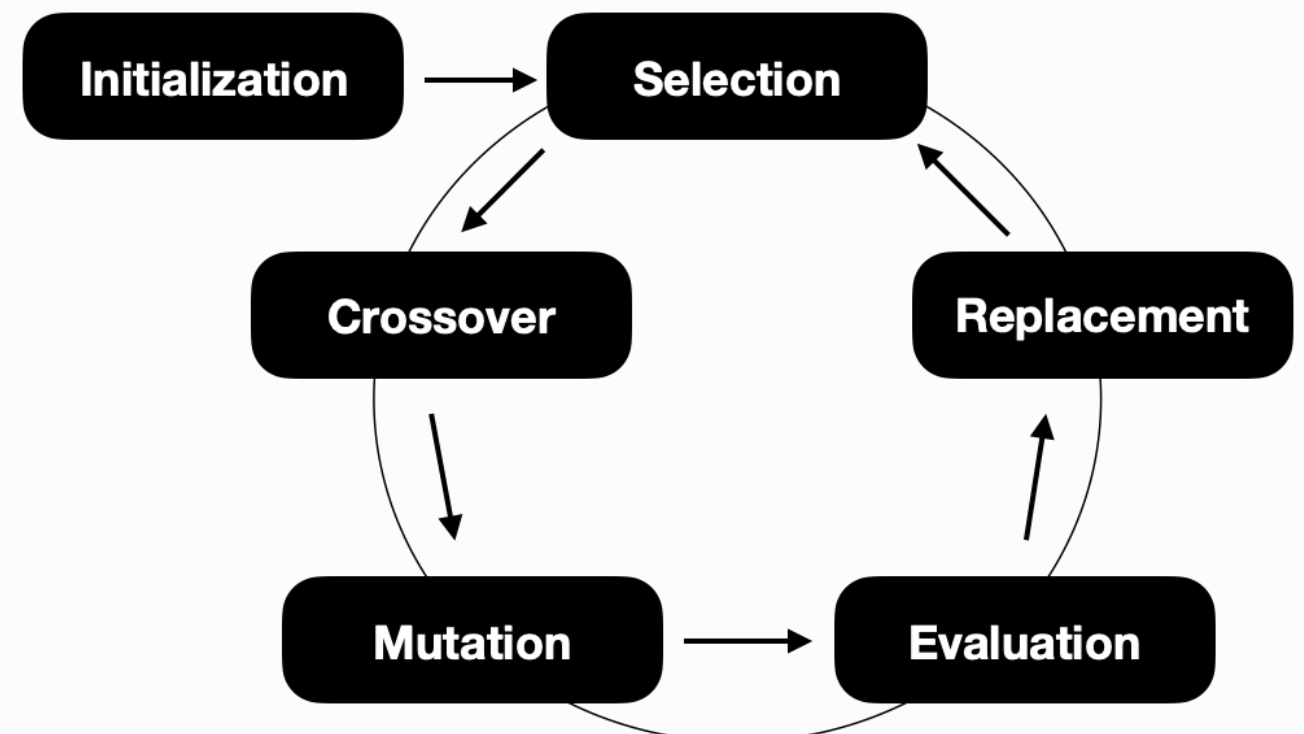
Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Partiamo da una delle tipiche applicazioni degli algoritmi genetici.

Problema: Prima dell'inizio di ogni semestre, il Dipartimento di Informatica ha il compito di redigere l'orario delle lezioni, coordinando le necessità di studenti e docenti, oltre che rispettando vincoli connessi al tempo da dedicare ad ogni corso e le aule utilizzabili e la loro capienza.

Esercizio: Definire la codifica degli individui, oltre che la funzione di fitness e gli operatori genetici per questo problema. Avete 15 minuti, poi lo discuteremo insieme.



Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Partiamo da una delle tipiche applicazioni degli algoritmi genetici.

Problema: Prima dell'inizio di ogni semestre, il Dipartimento di Informatica ha il compito di redigere l'orario delle lezioni, coordinando le necessità di studenti e docenti, oltre che rispettando vincoli connessi al tempo da dedicare ad ogni corso e le aule utilizzabili e la loro capienza.

Esercizio: Definire la codifica degli individui, oltre che la funzione di fitness e gli operatori genetici per questo problema. Avete 15 minuti, poi lo discuteremo insieme.

Individuo: Una matrice 5xM, dove 5 rappresenta il numero di giorni della settimana utilizzabili ed M il numero di aule da allocare nel calendario.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Day 0	[2]	[2]	[2]	[0]	[0]	[6]	[6]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
Day 1	[0]	[0]	[49]	[49]	[0]	[0]	[0]	[0]	[10]	[10]	[0]	[0]	[0]	[0]	[0]
Day 2	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
Day 3	[0]	[0]	[0]	[0]	[0]	[0]	[19]	[19]	[19]	[0]	[0]	[0]	[0]	[0]	[0]
Day 4	[0]	[33]	[33]	[33]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
Day 5	[0]	[0]	[0]	[0]	[44]	[44]	[44]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]

Questo implica che una popolazione sarà formata da un insieme di matrici di dimensione 5xM.

La popolazione verrà inizializzata casualmente.

L'ij-esima cella verrà riempita con un ID che rappresenta il corso.

Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Partiamo da una delle tipiche applicazioni degli algoritmi genetici.

Problema: Prima dell'inizio di ogni semestre, il Dipartimento di Informatica ha il compito di redigere l'orario delle lezioni, coordinando le necessità di studenti e docenti, oltre che rispettando vincoli connessi al tempo da dedicare ad ogni corso e le aule utilizzabili e la loro capienza.

Esercizio: Definire la codifica degli individui, oltre che la funzione di fitness e gli operatori genetici per questo problema. Avete 15 minuti, poi lo discuteremo insieme.

Individuo: Una matrice $5 \times M$, dove 5 rappresenta il numero di giorni della settimana utilizzabili ed M il numero di aule da allocare nel calendario.

Fitness: Dobbiamo ottimizzare il calendario rispettando i requisiti. Pertanto, dovremo innanzitutto imparare a «calcolare» quanto una soluzione rispetta i requisiti.

$f(x) = \# \text{ corsi assegnati};$

$f(x) = \# \text{ corsi assegnati ad aule con capienza adeguata } (\# \text{ studenti} \leq \# \text{ posti});$

$f(x) = \# \text{ aule utilizzate};$

$f(x) = \# \text{ ore di corso per giorno};$

→ Ulteriori requisiti possono essere aggiunti, chiaramente, sulla base delle necessità.

Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Partiamo da una delle tipiche applicazioni degli algoritmi genetici.

Problema: Prima dell'inizio di ogni semestre, il Dipartimento di Informatica ha il compito di redigere l'orario delle lezioni, coordinando le necessità di studenti e docenti, oltre che rispettando vincoli connessi al tempo da dedicare ad ogni corso e le aule utilizzabili e la loro capienza.

Esercizio: Definire la codifica degli individui, oltre che la funzione di fitness e gli operatori genetici per questo problema. Avete 15 minuti, poi lo discuteremo insieme.

Selezione: Qualsiasi algoritmo è potenzialmente adatto per selezionare gli individui che entreranno nel mating pool.

Crossover: Qui dobbiamo stare attenti. Quali sono le intersezioni ammissibili?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Day 0	[2]	[2]	[2]	[0]	[0]	[6]	[6]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
Day 1	[0]	[0]	[49]	[49]	[0]	[0]	[0]	[0]	[10]	[10]	[0]	[0]	[0]	[0]	[0]
Day 2	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
Day 3	[0]	[0]	[0]	[0]	[0]	[0]	[19]	[19]	[19]	[0]	[0]	[0]	[0]	[0]	[0]
Day 4	[0]	[33]	[33]	[33]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]
Day 5	[0]	[0]	[0]	[0]	[44]	[44]	[44]	[0]	[0]	[0]	[0]	[0]	[0]	[0]	[0]

Abbiamo due opzioni possibili. O tagliamo per righe la matrice, così da mischiare le soluzioni in termini di giorni...

Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

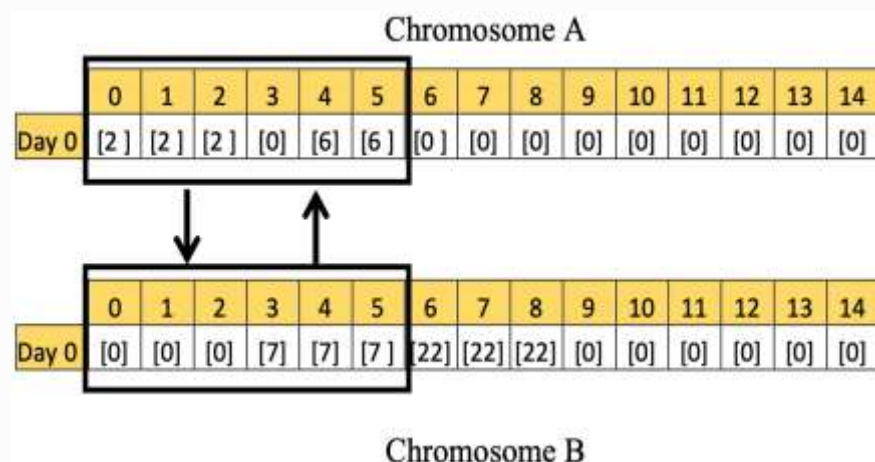
Partiamo da una delle tipiche applicazioni degli algoritmi genetici.

Problema: Prima dell'inizio di ogni semestre, il Dipartimento di Informatica ha il compito di redigere l'orario delle lezioni, coordinando le necessità di studenti e docenti, oltre che rispettando vincoli connessi al tempo da dedicare ad ogni corso e le aule utilizzabili e la loro capienza.

Esercizio: Definire la codifica degli individui, oltre che la funzione di fitness e gli operatori genetici per questo problema. Avete 15 minuti, poi lo discuteremo insieme.

Selezione: Qualsiasi algoritmo è potenzialmente adatto per selezionare gli individui che entreranno nel mating pool.

Crossover: Qui dobbiamo stare attenti. Quali sono le intersezioni ammissibili?



O, altrimenti, per colonna. Ma in questo caso dovremo assicurarci di scambiare i cromosomi mantenendo l'ordine delle colonne – se così non fosse, perderemmo la coerenza delle informazioni sulle aule!

Algoritmi Genetici

Algoritmo genetico per il problema delle 8 regine

Per affrontare il problema è necessario definire la **codifica degli individui**, la **funzione di fitness** e gli **operatori genetici**.

Selezione: qualunque algoritmo di selezione può essere usato per scegliere gli individui che entreranno nel *mating pool*.

Crossover: va progettato con attenzione, per garantire intersezioni ammissibili e quindi soluzioni valide.

Mutazione: deve produrre individui ancora validi. Può essere applicata a livello di singola cella (cambiando la posizione di una regina) oppure in forma globale (scambiando colonne o righe).

Il problema può essere trattato come **multi-obiettivo**, ma non tutti i criteri hanno lo stesso peso. Possiamo usare il *preference sorting* per stabilire priorità, oppure trasformarlo in un problema **mono-obiettivo** con una funzione che combina gli obiettivi tramite una **somma pesata e normalizzata**, dando più importanza ad alcuni fattori. Infine, è essenziale avere una **base di conoscenza** con dati accurati (ad esempio ID, vincoli e risorse disponibili). Senza una buona analisi dei requisiti e una ricognizione preliminare, l'algoritmo non può essere efficace.

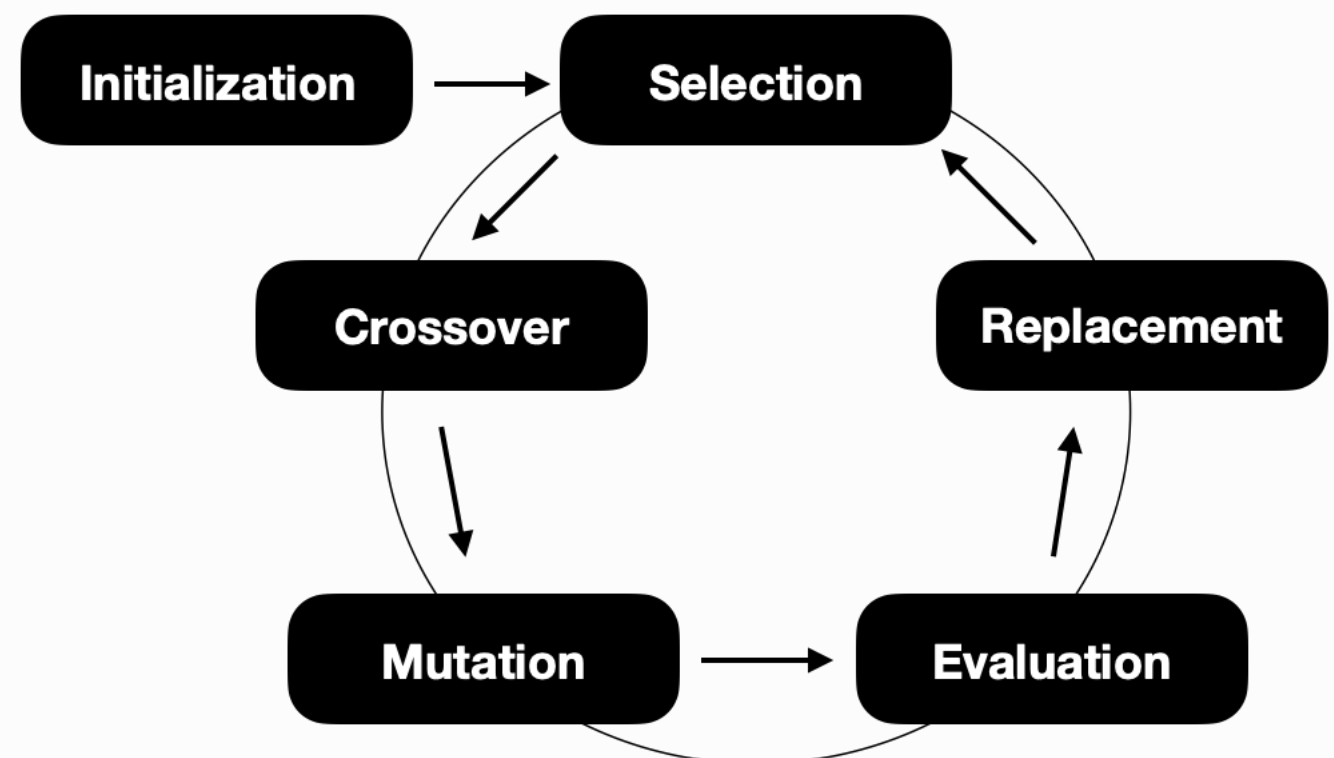
Algoritmi Genetici

Consideriamo adesso un problema completamente diverso...

Partiamo da un caso «banale» per arrivare poi a qualcosa di più complesso.

Problema: Vogliamo ottenere la scritta «Hello World!» con un algoritmo genetico. NB: è una istanza di un problema di generazione di testi, tanto per dire...

Esercizio: Definire la codifica degli individui, oltre che la funzione di fitness e gli operatori genetici per questo problema. Avete 15 minuti, poi lo discuteremo insieme.



Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Risolviamo un ultimo problema con gli algoritmi genetici.

Problema: Immaginiamo che l'anno accademico sia al termine. E' fine luglio, fa caldo e tra qualche giorno partiremo insieme al partner, direzione Gran Canaria. C'è solo una compagnia aerea che consente di arrivare da Roma a Gran Canaria, la quale prevede una commissione aggiuntiva di 150 euro per i bagagli a mano che superano i 10kg.

Per evitare di pagare questa commissione, vogliamo perciò ottimizzare la composizione del nostro zaino. Vogliamo massimizzare il numero di elementi nello zaino, tenendo però in considerazione il fatto che dobbiamo necessariamente portare con noi il nostro Macbook di 2kg.

Possiamo scegliere di portare ciascuno dei seguenti elementi: infradito, beauty case, libri, un ricambio. Laddove sia necessario scegliere, vogliamo preferire l'inserimento nello zaino del beauty case.

Esercizio: Definire la codifica degli individui, la procedura di inizializzazione della popolazione, la funzione di fitness, eventuali vincoli del problema, e operatori genetici.

Le caratteristiche degli elementi sono volutamente lasciate alla personale interpretazione. Usate la testa: gli infradito non possono pesare più del beauty case!

Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Risolviamo un ultimo problema con gli algoritmi genetici.

Problema: Immaginiamo che l'anno accademico sia al termine. E' fine luglio, fa caldo e tra qualche giorno partiremo insieme al partner, direzione Gran Canaria. C'è solo una compagnia aerea che consente di arrivare da Roma a Gran Canaria, la quale prevede una commissione aggiuntiva di 150 euro per i bagagli a mano che superano i 10kg.

Codifica: Potrebbe essere ragionevole usare una codifica binaria, in cui uno 0 rappresenta l'assenza dell'elemento nello zaino, 1 la sua presenza. Tuttavia, potremmo considerare di portare più libri con noi: pertanto, una codifica fatta come array di interi potrebbe essere una scelta più opportuna → In assenza di indicazioni più specifiche sul problema, entrambe le codifiche potrebbero funzionare.

Ma un secondo. A prescindere dalla codifica che scegliamo, cosa rappresenta un individuo?

Possiamo avere array di quattro elementi (che rappresentano presenza/assenza o numero di infradito, beauty case, libri e ricambio nello zaino) oppure considerare un array di cinque elementi (prevedendo il Macbook).

Dato il problema, la seconda soluzione NON è accettabile. Perché?

Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Risolviamo un ultimo problema con gli algoritmi genetici.

Problema: Immaginiamo che l'anno accademico sia al termine. E' fine luglio, fa caldo e tra qualche giorno partiremo insieme al partner, direzione Gran Canaria. C'è solo una compagnia aerea che consente di arrivare da Roma a Gran Canaria, la quale prevede una commissione aggiuntiva di 150 euro per i bagagli a mano che superano i 10kg.

Vincoli: Dobbiamo necessariamente portare con noi il Macbook, per cui ogni soluzione dovrà rispettare questo vincolo.

O meglio, la codifica a cinque elementi può diventare accettabile SE E SOLO SE teniamo in considerazione il vincolo durante le successive fasi dell'algoritmo.

Procedura di inizializzazione: Una procedura pseudo-casuale che prevede l'inserimento di un «1» nella cella dedicata al Macbook nel caso di una codifica a cinque elementi. Una procedura pseudo-casuale che sia ragionevole – non ha senso generare soluzioni che prevedono un valore «14» per gli infradito! Possiamo, ad esempio, vincolare l'inizializzazione assegnando un range valido per ogni elemento. Nel caso in cui la codifica è binaria, invece, possiamo limitarci ad una codifica pseudo-casuale che tenga solo in conto del vincolo.

Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Risolviamo un ultimo problema con gli algoritmi genetici.

Problema: Immaginiamo che l'anno accademico sia al termine. E' fine luglio, fa caldo e tra qualche giorno partiremo insieme al partner, direzione Gran Canaria. C'è solo una compagnia aerea che consente di arrivare da Roma a Gran Canaria, la quale prevede una commissione aggiuntiva di 150 euro per i bagagli a mano che superano i 10kg.

Fitness: Ci interessa massimizzare il numero di elementi nello zaino. Al tempo stesso, dobbiamo rimanere sotto i 10kg, per cui dobbiamo minimizzare il peso.

$f(x) = \text{\#elementi inseriti};$

MAX

$f(x) = \text{peso dello zaino};$

MIN

Attenzione però, il problema ci diceva di preferire soluzioni che contenessero il beauty case! Pertanto, in questo problema multi-obiettivo dovremmo prevedere una funzione di preferenza, del tipo:

$p(x) = \text{soluzione contenente il beauty case};$

NB: Anche senza conoscere le caratteristiche del problema, riusciamo a formularlo senza problemi!

Algoritmi Genetici

Consideriamo adesso, insieme, dei problemi più reali di quello delle 8 regine

Risolviamo un ultimo problema con gli algoritmi genetici.

Problema: Immaginiamo che l'anno accademico sia al termine. E' fine luglio, fa caldo e tra qualche giorno partiremo insieme al partner, direzione Gran Canaria. C'è solo una compagnia aerea che consente di arrivare da Roma a Gran Canaria, la quale prevede una commissione aggiuntiva di 150 euro per i bagagli a mano che superano i 10kg.

Selezione: Qualsiasi algoritmo è potenzialmente adatto per selezionare gli individui che entreranno nel mating pool.

Crossover: Possiamo intrecciare le soluzioni a piacimento → Il vincolo è sempre rispettato. Perché?

Abbiamo vincolato l'inizializzazione della popolazione!

Mutazione: Qui dobbiamo stare più attenti, poiché una mutazione completamente casuale potrebbe violare il vincolo. Quindi, l'operatore può mutare casualmente tutte le celle dell'array ad eccezione di quella relativa al Macbook.

Et voilà, esercizio concluso!